UPA App: Fachliches Funktionskonzept

Vorbereitung Dozentengespraech | Einzeltermin Gruppe fuer Gruppe (Scope-Abgleich)

1. Architektur & Begruendung: Python (FastAPI) statt Java

Termintreue & MVP-Fokus:	Python spart ca. 60% Boilerplate gegenueber Java. Die Entwicklungszeit fliesst voll in Business-Logik und den Android-Client statt in Framework-Overhead.
Schnittstellen-Effizienz:	Asynchrone Web-Bibliotheken (httpx, google-api-client) binden Google Workspace Admin SDK, Classroom und Instagram unuebertroffen schlank an.
Automatische API-Doku:	FastAPI generiert nativ interaktive OpenAPI/Swagger-Docs (/docs) zur direkten Schnittstellenabstimmung mit dem Android Studio Team.

2. Funktionale Struktur der App (Module & Masken)

A. Modul: Inicio (Campus-Feed)

* **Dringlichkeits-Eilmeldungen:** Temporaere Warnbanner oben im Feed (z.B. Unterrichtsausfall bei Hagel/Unwetter).
* **Campus-Events & Charlas:** Vortraege und Veranstaltungen mit nativer Ein-Klick-An- und Abmeldung fuer Studenten.
* **Social Media Integration:** Automatisierter Beitrags-Feed der universitaeren Kanaele via SociableKIT / Instagram API.

B. Modul: Agenda (Termine & Fristen)

* **Kombinierte Ansicht:** Chronologische Terminliste oben, synchronisierte Monatsuebersicht mit Terminpunkten unten.
* **Google Classroom Synchronisation:** Automatische Uebernahme akademischer Hausaufgaben- und Abgabefristen.

C. Modul: Profil & Personalisierung

* **Google OAuth Domaenen-Sperre:** Zwangsanmeldung mit @upa.edu.py. Zugriffsschutz ohne eigenen Mailserver.
* **Benutzer-Praeferenzen:** Einstellung von Dark/Light Mode, Studiengangszuordnung und Benachrichtigungsfiltern.

D. Modul: Web-Administration (Marketing-Portal)

* **Integriertes Datenmodell:** Single Source of Truth: Ein Eintrag kann Beitragsnachricht, Kalendertermin oder beides sein.
* **Zielgruppen-Segmentierung:** Aussteuerung nach Studiengang/Rolle basierend auf Google Groups via Google Admin API.